

Checklist Mingguan — database reference

As of 6 October 2026, up to and including migration `20261006010000_hr_administers.sql`.

This document was written from the schema itself: all 41 migrations were applied to a real Postgres (the PGLite test harness) and the catalog was read back. The live database matches it — `supabase/tests/verify_policies.sql` ran on the live project on 6 Oct 2026 and every check passed. When the two disagree, the migrations win; update this file (see [Keeping this current](#)).

Contents

- 1. [The server](#)
- 2. [Sign-in and accounts \(Auth\)](#)
- 3. [Who may do what](#)
- 4. [Tables](#)
- 5. [Views](#)
- 6. [Functions](#)
- 7. [Triggers](#)
- 8. [File storage](#)
- 9. [Edge Functions](#)
- 10. [Scheduled jobs](#)
- 11. [Admin scripts](#)
- 12. [Migration history](#)
- 13. [Keeping this current](#)

1. The server

Everything runs on one **Supabase** project. There is no other server.

Item	Value
Project ref	<code>aorkigafuepkexymexjt</code>
API URL	<code>https://aorkigafuepkexymexjt.supabase.co</code>
Dashboard	<code>https://supabase.com/dashboard/project/aorkigafuepkexymexjt</code>
Database	PostgreSQL 15 or later (the views need <code>security_invoker</code> , added in 15)
Time zone	The server runs on UTC. Every "today" in the reports uses <code>report_today()</code> , which is Malaysia's date (UTC+8).

What lives in the project:

Part	Used for
Database (<code>public</code> schema)	All app data: 24 tables, 17 views, 39 functions, 8 triggers, 49 row-level security policies.

Part	Used for
Auth	One login per person, keyed on the payroll number. See section 2.
Storage	One private bucket, <code>return-photos</code> , for damage-return evidence. See section 8.
Edge Functions	<code>payroll-auth</code> , <code>xlsx-export</code> , <code>purge-return-photos</code> . See section 9.
Extensions	<code>pgcrypto</code> (password hashing, in the <code>extensions</code> schema), <code>pg_cron</code> and <code>pg_net</code> (the nightly photo purge), Vault (holds the service-role key for that job).

Who connects, and with what:

Client	Key	What it can reach
Phone app (Vercel web build)	Anon key + the signed-in person's session token	Only what row-level security lets that person see. The anon key is public by design.
Reports web app	Same as the phone app	Same — the <code>report_*</code> views run under the caller's own permissions.
Edge Functions	Service-role key (from the Supabase environment) or the caller's token	<code>payroll-auth</code> and <code>purge-return-photos</code> use the service role; <code>xlsx-export</code> uses the caller's token.
Supabase SQL editor	Project owner	Everything, past row-level security. This is where migrations and admin scripts are run.

Secrets are never in the repository. The service-role key lives in the Edge Function environment (set by Supabase) and in Vault under the name `service_role_key`. The starting password lives in the `login_settings` table. The phone app's `.env.local` holds only the URL and the anon key.

How changes reach the live database. There is no automatic deploy. Each new file in `supabase/migrations/` is pasted into the SQL editor and run, in filename order, once. Then `supabase/tests/verify_policies.sql` is run, and every row must read PASS. Edge Functions are deployed from the repository root with `npx supabase functions deploy <name>`.

2. Sign-in and accounts (Auth)

How a person signs in

1. The person types their **payroll number** (for example `KP0093`) and password into the app.
2. The app calls the `payroll-auth` Edge Function with them.
3. `payroll-auth` looks the payroll number up in `users` (with the service role) and works out the login address: the person's real e-mail if `users.email` is set, otherwise `kp0093@checklist.local`. The address never leaves the function.
4. It signs in to Supabase Auth with that address and the password, and returns the session tokens to the app.
5. From then on every request carries the session token. The database knows the caller through `auth.uid()`, which matches `users.auth_user_id`.

General Manager accounts are turned away by the phone app after sign-in; they use the reports web app.

How a login is created

- **Automatically.** When a row is inserted into `users` (from the app or SQL), the trigger `users_provision_login` calls `provision_login()`. It creates the Auth account, sets the password to the **starting password** stored in `login_settings`, and writes the new account's id into `users.auth_user_id`. Reactivating someone who never had a login does the same.
- If `login_settings` has no row, adding a person fails with a message saying so. That is deliberate: a person who exists but cannot sign in is the failure this prevents.
- `supabase/provision_logins.sql` is the manual backfill for rows created before the trigger existed.

Keeping the login address current

`users_sync_auth_email / sync_auth_email()` copies `users.email` (or the `@checklist.local` address when there is none) onto the Auth account whenever the e-mail, the payroll number or the link changes. A person may change only their own e-mail, through `set_my_email()`.

Passwords

Situation	What happens
New account	Starts on the password in <code>login_settings</code> . The person changes it under My account after their first sign-in.
Person changes their own	My account → Change password in the app (at least 6 characters).
Forgotten	Admin runs <code>supabase/reset_password.sql</code> (payroll number + new password). It also signs out every phone still holding the old session. If the person has a real e-mail, <code>payroll-auth</code> 's <code>request-reset</code> can send Supabase's reset mail instead.

Session limits (enforced by the app)

The app signs a person out after **15 minutes** without a touch, or **3 hours** after signing in (`marks-app/src/data/sessionLimits.ts`).

3. Who may do what

The database, not the app, decides what each person can see and change. Every table has **row-level security** on, and every policy is built from a small set of helper functions (section 6) that look the caller up in `users`.

Outlet reach. Most people see only their own outlet (`users.branch_id`). An Area Manager also sees every outlet listed for them in `user_branches`. Manager, General Manager, HR and Admin hold no outlet and see all of them.

Role	Sees	Writes
Shop Staff (staff)	People, marks and returns at their outlet.	Nothing but their own e-mail.
Store Staff / Store Clerk (store , clerk)	People and marks at HQ; every outlet's returns.	Returns, return stages, photos, new suppliers.
Supervisor (supervisor , SV or Asisten)	People, marks and returns at their outlet.	Marks for shop and store staff; adds Shop Staff; asset issues at their outlet.
Area Manager (area_manager)	The same, at every outlet they cover; plus Tugasan.	Marks for supervisors; verifies marks; Tugasan; asset issues; reminders; adds Shop Staff and Supervisors; tags SV/Asisten.
Manager (manager)	Every outlet, kedai side only — no returns, no store marks.	As an Area Manager, over every outlet.
General Manager (general_manager)	Everything, including returns.	Marks, verifications, reminders and returns anywhere. Set only through SQL.
HR (human_resources) and Admin (admin)	Everything except returns.	Accounts, roles, outlets, change logs, checklist reference data, scoring rules, SV/Asisten titles. As head office the database also lets them correct marks and verifications, though the app gives them no screen for it. Since 6 Oct 2026 HR has exactly Admin's rights.

What the app shows is narrower than what the database allows. For example, Shop Staff can read every mark at their outlet at the database level, but the app only ever shows them their own. The database rules above are the security boundary; the app's screens are a choice of what to display.

Table privileges. Row-level security only filters rows; the signed-in role also needs table privileges. **authenticated** is granted select, insert, update and delete on every table and view, except:

- **assets** — read only (its state is kept by a trigger).
- **asset_issues** — read; insert only **asset_id**, **note**, **opened_on**; update only **note**, **resolved_on**; no delete.
- **login_settings** — no access at all.

anon (signed out) is granted nothing.

4. Tables

24 tables. All have row-level security on. Payroll numbers (**users.id**) are referenced with **ON UPDATE CASCADE**, so changing a number carries every record with it.

People and outlets

Table	Holds	Key rules
<code>branches</code>	Outlets. <code>id</code> is the short code (<code>DMC</code>), 2–5 capitals or digits, permanent. <code>active = false</code> closes an outlet.	Read by anyone signed in. Written by Admin/HR. A new row automatically gets its <code>scoring_rules</code> row and its ten <code>assets</code> rows.
<code>users</code>	Every person. <code>id</code> is the payroll number (2–12 capitals/digits). Also <code>name</code> , <code>short_name</code> , <code>initials</code> , <code>role</code> , <code>branch_id</code> (null for head office), <code>active</code> , <code>joined_on</code> , <code>auth_user_id</code> , <code>email</code> , <code>supervisor_title</code> .	Read: Admin/HR, yourself, and people at outlets you reach. Written by Admin/HR. A Supervisor may insert Shop Staff, an Area Manager Shop Staff or Supervisors, at outlets they cover. <code>supervisor_title</code> only on supervisors.
<code>user_branches</code>	Extra outlets an Area Manager covers, beyond <code>users.branch_id</code> .	Only Area Managers may have rows (trigger). Written by Admin/HR.
<code>role_changes</code>	Every promotion, demotion and transfer: who, from, to, by whom, when.	Admin/HR only.
<code>branch_changes</code>	Every change of outlet.	Admin/HR only.
<code>payroll_id_changes</code>	Every change of payroll number.	Admin/HR only.
<code>login_settings</code>	One row: the starting password for new logins (at least 6 characters).	No app access; read only by <code>provision_login()</code> . Set in the SQL editor.

Checklist reference data

Table	Holds
<code>checklist_forms</code>	The three forms: <code>kedai</code> (Shop Staff, 22 perkara), <code>stor</code> (Store Staff, 17), <code>sv</code> (Supervisor, 19). One form per role.
<code>checklist_categories</code>	The numbered categories of each form.
<code>checklist_lines</code>	One scorable perkara each.
<code>scoring_rules</code>	Per outlet: pass mark (80), scale (5 or 10; 5 today), and whether the Area Manager must verify.
<code>tugasan_items</code>	The Area Manager's self-check items (<code>peti_cash</code> , <code>x_report</code>).
<code>suppliers</code>	Suppliers returns go back to.

Read by anyone signed in (scoring rules: by those who reach the outlet); written by Admin/HR. The store team and the General Manager may also add a supplier.

Marks

Table	Holds	Key rules
marks	One weekly score per person: user_id , branch_id , form_key , year, month, week_no (1–4), total_score , max_score , note , scored_by . One per person per week.	pct is generated from $\text{total} \div \text{max}$, never typed. branch_id and form_key are a snapshot of where the person was when marked. Readable by whoever reaches the outlet (store marks hidden from the Manager). Written by those allowed to score. Locked once verified .
mark_lines	The score for each perkara (0 up to the scale).	A perkara marked N/A has no row and does not count in max_score ; 0 counts. Locked with its mark.
mark_verifications	The Area Manager's confirmation, optionally with adjusted_to (a corrected total).	Absence means "not yet verified".

Area Manager's Tugasan

Table	Holds
tugasan_checks	Per outlet × month × week × item: done, note (e.g. cash amount), date inspected.
tugasan_signoffs	Per outlet × month × week: filled by, checked by, date signed.

Read by Area Managers/Managers of that outlet and head office; written by Area Managers/Managers and Admin/HR. Staff and supervisors cannot read them.

Shop assets (Aset kedai)

Table	Holds	Key rules
assets	The fixed list per outlet: A) AIR-COND ... J) LAIN-LAIN. Plus a summary: is_open , note , opened_on , resolved_on .	Read only to the app. The summary columns are kept up to date by a trigger from asset_issues .
asset_issues	One row per reported problem: note , opened_on , opened_by , resolved_on , resolved_by . Several may be open on the same asset.	Supervisors, Area Managers, Managers and head office at that outlet may report and resolve. Who resolved it is stamped by the database. Never deleted from the app.

Returns (pulangan)

Table	Holds	Key rules
returns	One bill being returned: ref (PR0001), outlet, bill number and date, reason (damage/expired), remark, supplier, disposition (supplier/discard).	Read by the store team (all outlets), anyone posted at or covering that outlet, and the General Manager — not the Manager, Admin or HR. Written by the store team and the General Manager.
return_events	One row per stage reached, with its date: received → submitted to clerk → segregated → supplier called → picked up / discarded → adjusted.	Same visibility as the bill.
return_photos	Up to 2 photos per return; the file itself is in Storage.	Kept until 30 days after the bill is adjusted, then purged (section 10).

Reminders

Table	Holds	Key rules
reminders	An in-app nudge from an Area Manager/Manager (or head office) to a Supervisor about unmarked staff.	The recipient may only mark it read.

Enumerated types

Type	Values
user_role	staff, store, clerk, supervisor, area_manager, manager, general_manager, human_resources, admin
supervisor_title	sv, asisten
return_reason	damage, expired
return_disposition	supplier, discard
return_stage	received, submitted_to_clerk, segregated, supplier_called, picked_up, discarded, adjusted
tugasan_note_kind	amount, status

5. Views

All 17 views are **security_invoker**: they run with the **caller's** permissions, so a view never shows a row its underlying table would hide.

For the reports web app (**report_***):

View	One row per
report_periods	Month, oldest first to the current month, with how many checklist weeks have started.

View	One row per
<code>report_due</code>	Active person × month they are due a mark.
<code>report_marks</code>	Mark, with the pass rule applied; <code>final_pct</code> is what the phone app shows.
<code>report_staff_monthly</code>	Person × month × form: the four weekly percentages, average, passes, verifications, SV/Asisten title.
<code>report_branch_weekly</code>	Outlet × week × form: headcount, marked, gaps, passed.
<code>report_branch_monthly</code>	Outlet × month × form: the same, for the month.
<code>report_company_weekly</code> / <code>report_company_monthly</code>	The whole company, summed from the outlet rows.
<code>report_tugasan_branch_monthly</code>	Outlet × month: Tugasan weeks filled and checked.
<code>report_returns_branch_monthly</code>	Outlet × month received: returns received, on time, open, breach, overdue, average turnaround.
<code>report_returns_open</code>	Every return not yet adjusted, with its ageing status and last stage.

Returns KPI building blocks:

View	Shows
<code>return_turnaround</code>	Days from received to adjusted (open bills age against today).
<code>return_submission</code>	Each return's Friday hand-over deadline and whether it was met.
<code>return_submission_kpi</code>	The hand-over rate summarised.
<code>return_ageing</code>	<code>ok</code> , <code>breach</code> (past 60 days, inside the 7-day window to clear) or <code>overdue</code> .
<code>return_stage_gaps</code>	Days spent between each stage, for bottlenecks.

Other: `mark_coverage` — which person × week cells have no mark.

6. Functions

Security helpers. Each answers one question about the signed-in person. They are `SECURITY DEFINER` (they read `users` past row-level security) and every policy is built from them — change a rule here, not in twenty policies.

Function	Answers
<code>app_user_id()</code> , <code>app_role()</code> , <code>app_branch_id()</code>	The caller's payroll number, role and home outlet.
<code>app_is_admin()</code>	Is the caller Admin or HR? (the administration console)

Function	Answers
<code>app_is_exec()</code>	General Manager, HR or Admin?
<code>app_is_cross_branch()</code>	A head-office role that sees every outlet?
<code>app_manages_outlets()</code>	Area Manager or Manager?
<code>app_can_see_branch(outlet)</code>	May the caller see this outlet?
<code>app_can_see_mark(outlet, form)</code>	Outlet reach plus: store marks hidden from the Manager.
<code>app_can_score()</code>	May the caller write marks?
<code>app_mark_verified(mark)</code>	Is this mark verified (and so locked)?
<code>app_can_read_tugasan(outlet)</code>	May the caller read this outlet's Tugasan?
<code>app_can_see_store_ops()</code>	May the caller see store-side marks? (everyone but the Manager)
<code>app_can_see_returns(), app_is_returns_writer(), app_is_central_store(), app_can_see_branch_returns(outlet)</code>	The returns rules.

Called by the app:

Function	Does
<code>set_my_email(new_email)</code>	Lets anyone change their own e-mail, and nothing else on their row.
<code>set_supervisor_title(person, title)</code>	Admin/HR, or the Area Manager over that outlet, tags a supervisor <code>sv</code> or <code>asisten</code> .

Report and returns arithmetic: `report_today()` (Malaysia's date), `report_week_of(date)` (checklist week 1–4: days 1–7, 8–14, 15–21, 22–end), `report_first_week(...)` (the week a new person is first due), `return_due_on(received)` (the Friday hand-over deadline; weekend arrivals roll to the next week), `return_age_limit_days()` (60), `return_grace_days()` (7), `stage_rank(stage)`.

Used by triggers and jobs: `provision_login()`, `sync_auth_email()`, `branches_default_assets(outlet)` and `asset_catalogue()` (the ten asset rows), `assets_refresh_summary(asset)`, `return_photos_due_for_purge(days, grace)` (service role only), plus the trigger functions in section 7.

7. Triggers

Table	Trigger	Does
users	users_provision_login	After insert, or on reactivation: creates the person's login on the starting password.
users	users_sync_auth_email	Keeps the login address in step with the e-mail and payroll number.
user_branches	user_branches_role_check	Refuses extra outlets for anyone who is not an Area Manager.
branches	branches_default_scoring_rule	Gives a new outlet its scoring rules.
branches	branches_default_assets	Gives a new outlet its ten asset rows.
asset_issues	asset_issues_stamp_resolver	Records who resolved an issue; a resolve date earlier than the open date is moved up to it.
asset_issues	asset_issues_refresh_summary	Recomputes the asset row's open/closed summary.
return_photos	return_photos_cap_check	Refuses a third photo on one return.

8. File storage

Bucket	Settings	Access
return-photos	Private. 1 MB per file. JPEG or WebP only. Path: <code><outlet>/<return_ref>/<file></code> . At most 2 per return.	Storage policies follow the returns rules: whoever may see the return may see its photos.

Files are never deleted with SQL — Supabase blocks it. The app and the purge job remove the file through the Storage API first, then the `return_photos` row.

9. Edge Functions

All three live in `supabase/functions/` and are deployed with `npm run supabase functions deploy <name>` from the repository root.

Function	Called by	Runs as	Does
payroll-auth	The sign-in screen	Service role (lookup), then a normal sign-in	<code>sign-in</code> : turns a payroll number into the login address and signs in. <code>request-reset</code> : sends Supabase's password-reset mail; always answers "ok", so it cannot reveal who is on the payroll.
xlsx-export	Export button on the Area Manager home	The caller's own session	Builds the month's marks workbook from exactly what the caller can see.

Function	Called by	Runs as	Does
<code>purge-return-photos</code>	The nightly job, or an Admin by hand	Service role or an active Admin	Removes photos 30 days after their bill was adjusted, plus orphan files. <code>{"dry_run": true}</code> lists without deleting. Refuses fewer than 30 days.

10. Scheduled jobs

Job	When	Does
<code>purge-return-photos</code> (<code>pg_cron</code> , job id 1)	Daily, 19:00 UTC (03:00 Malaysia)	<code>pg_net</code> calls the Edge Function with the service-role key from Vault. Set up by <code>supabase/schedule_photo_purge.sql</code> .

To check it ran:

```
SELECT status, return_message, start_time FROM cron.job_run_details ORDER BY
start_time DESC LIMIT 5;
```

11. Admin scripts

These are kept in `supabase/` and run by hand in the SQL editor. They are not migrations.

Script	Purpose
<code>reset_password.sql</code>	Give one person a new password and sign out their phones.
<code>provision_logins.sql</code>	Create logins for any active person who has none.
<code>import_marks.sql</code>	Load weekly marks per perkara for many people at once, with every row checked first.
<code>xlsx_to_import_marks.py</code>	Turn the old Excel workbooks into rows for <code>import_marks.sql</code> .
<code>schedule_photo_purge.sql</code>	One-time setup of the nightly photo purge.
<code>seed.sql</code>	Sample and workbook data for a fresh test database. Never run on live.

12. Migration history

Run in this order. Each file explains itself in its opening comment.

Migration	Change
<code>20260909010000_init</code>	The schema: tables, views, constraints.
<code>20260909010100_auth_bridge</code>	Links Supabase Auth to the staff directory; the security helpers.

Migration	Change
20260909010200_rls	Row-level security: outlet scoping enforced by the database.
20260909020000_return_submission_stage	Store staff hand each return list to the clerk before Friday.
20260909020100_return_kpi	The store-staff returns KPI.
20260909030000_tugasan_stays_with_area_manager	Tugasan is the Area Manager's own self-check.
20260909030100_returns_leave_admin	Admin is taken out of returns.
20260909030200_central_store_at_hq	The store team works at HQ and sees every outlet's returns.
20260909030300_grants	Table privileges for signed-in users.
20260909030400_real_branches	The real outlet list.
20260910010000_sv_checklist	The 19-perkara Supervisor checklist.
20260910010100_who_may_score	One rule for who may score.
20260910020000_return_photos	Photo evidence on damage returns.
20260910030000_mark_queries	A question thread on a mark (removed 19 Sep).
20260910030100_reminders	Reminders from Area Manager to Supervisor.
20260916010000_branch_staff_management	Supervisors and Area Managers add their own staff.
20260916020000_tighten_grants	Grants brought back to what was intended.
20260917010000_user_email	A real e-mail per person, for password resets.
20260917020000_set_my_email	A person may update their own e-mail.
20260917030000_service_role_reads_users	The service role may read the directory (for payroll-auth).
20260918010000_auto_provision_logins	A login is created the moment a person is added.
20260918020000_payroll_number_changes	A payroll number can change; history follows it.
20260918030000_manager_manages_outlets	The Manager acts as an Area Manager over every outlet.
20260918040000_report_views	The report_* views for the reports app.

Migration	Change
20260919010000_payroll_number_shape	Payroll numbers accept whatever payroll issues.
20260919020000_marks_open_until_verified	A mark can be changed until verified, then it is locked.
20260919030000_drop_mark_queries	The staff ↔ Supervisor question thread is removed.
20260919040000_assets_for_every_outlet	Every outlet gets the asset list.
20260919050000_area_manager_hires_supervisors	An Area Manager may add Supervisors too.
20260919060000_label_spelling	Two misspelt checklist labels corrected.
20260919070000_supervisor_title	SV vs Asisten Penyelia label.
20260919080000_photo_files_leave_through_storage	Photo files are removed through the Storage API.
20261002010000_tugasan_read_by_managers	Tugasan readable by those who fill or oversee it.
20261002020000_report_form_from_marks	A person-month is filed under the form they were marked on.
20261002030000_rls_once_per_query	Head-office access checked once per query (fixes report timeouts).
20261002040000_due_from_join_week	A new person is due from the week they were added.
20261003010000_malaysia_date	"Today" is Malaysia's date.
20261003020000_handover_before_friday	A return list before its Friday is not a missed hand-over.
20261003030000_marked_before_added	Someone marked before they were added is due all month.
20261003040000_asset_issues	Several open issues per asset, each resolved on its own.
20261006010000_hr_administers	HR has exactly Admin's rights; HR leaves returns.

Keeping this current

- **Tests.** `node supabase/tests/rls.test.mjs` applies every migration and the seed to a real Postgres and runs over 300 permission and report checks. Every new migration must be added to the lists in both `rls.test.mjs` and `gen_verify.mjs`.

- **Live check.** After adding a migration, run `node supabase/tests/gen_verify.mjs` to regenerate `verify_policies.sql`, then run that file on the live database.
- **This document.** Update the affected section and the migration history in the same change as the migration.
- `db/ERD.md` is the original design note from 9 Sep 2026. Its reasoning still holds, but this file is the current reference.